

GPT-5.5: The Complete Guide

OpenAI's Latest Model Deep Dive

Lucas Soares

June 2026

Table of Contents

Table of Contents

1. GPT-5 Overview & Timeline

Table of Contents

1. GPT-5 Overview & Timeline

2. Model Comparison & Performance

Table of Contents

1. GPT-5 Overview & Timeline

2. Model Comparison & Performance

3. Ok, great the model does well on a lot of benchmarks, but what is new about it?

Table of Contents

1. GPT-5 Overview & Timeline

2. Model Comparison & Performance

3. Ok, great the model does well on a lot of benchmarks, but what is new about it?

4. GPT-5 System Card

Table of Contents

- 1. GPT-5 Overview & Timeline**
- 2. Model Comparison & Performance**
- 3. Ok, great the model does well on a lot of benchmarks, but what is new about it?**
- 4. GPT-5 System Card**
- 5. Building with GPT-5: API & Integration**

Table of Contents

- 1. GPT-5 Overview & Timeline**
- 2. Model Comparison & Performance**
- 3. Ok, great the model does well on a lot of benchmarks, but what is new about it?**
- 4. GPT-5 System Card**
- 5. Building with GPT-5: API & Integration**
- 6. GPT-5 Prompting Guide Complete Breakdown**

Table of Contents

- 1. GPT-5 Overview & Timeline**
- 2. Model Comparison & Performance**
- 3. Ok, great the model does well on a lot of benchmarks, but what is new about it?**
- 4. GPT-5 System Card**
- 5. Building with GPT-5: API & Integration**
- 6. GPT-5 Prompting Guide Complete Breakdown**

GPT-5 Overview & Timeline

GPT-5 → GPT-5.5 Release Timeline

Version History (Aug 2025 → June 2026)

GPT-5 → GPT-5.5 Release Timeline

Version History (Aug 2025 → June 2026)

- **GPT-5** — Aug 2025 (original launch)

GPT-5 → GPT-5.5 Release Timeline

Version History (Aug 2025 → June 2026)

- **GPT-5** — Aug 2025 (original launch)
- **GPT-5.1** — Nov 2025

GPT-5 → GPT-5.5 Release Timeline

Version History (Aug 2025 → June 2026)

- **GPT-5** — Aug 2025 (original launch)
- **GPT-5.1** — Nov 2025
- **GPT-5.2** — Dec 2025 (reasoning_effort default → none; prompting guide deltas)

GPT-5 → GPT-5.5 Release Timeline

Version History (Aug 2025 → June 2026)

- **GPT-5** — Aug 2025 (original launch)
- **GPT-5.1** — Nov 2025
- **GPT-5.2** — Dec 2025 (reasoning_effort default → none; prompting guide deltas)
- **GPT-5.4** — Mar 2026 (mini + nano variants)

GPT-5 → GPT-5.5 Release Timeline

Version History (Aug 2025 → June 2026)

- **GPT-5** — Aug 2025 (original launch)
- **GPT-5.1** — Nov 2025
- **GPT-5.2** — Dec 2025 (`reasoning_effort` default → none; prompting guide deltas)
- **GPT-5.4** — Mar 2026 (mini + nano variants)
- **GPT-5.5** — Apr 2026 (current; default `reasoning_effort` reverts to medium)

GPT-5 → GPT-5.5 Release Timeline

Version History (Aug 2025 → June 2026)

- **GPT-5** — Aug 2025 (original launch)
- **GPT-5.1** — Nov 2025
- **GPT-5.2** — Dec 2025 (`reasoning_effort` default → none; prompting guide deltas)
- **GPT-5.4** — Mar 2026 (mini + nano variants)
- **GPT-5.5** — Apr 2026 (current; default `reasoning_effort` reverts to medium)
- **GPT-5.5-instant** — May 2026 (ChatGPT default)

GPT-5.5 Core Innovations

GPT-5.5 Core Innovations

- **Frontier Intelligence** — "a new class of intelligence for coding and professional work"

GPT-5.5 Core Innovations

- **Frontier Intelligence** — "a new class of intelligence for coding and professional work"
- **Explicit Model Selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano

GPT-5.5 Core Innovations

- **Frontier Intelligence** — "a new class of intelligence for coding and professional work"
- **Explicit Model Selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano
- **Tunable Reasoning** — none → low → medium → high → xhigh

GPT-5.5 Core Innovations

- **Frontier Intelligence** — "a new class of intelligence for coding and professional work"
- **Explicit Model Selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano
- **Tunable Reasoning** — none → low → medium → high → xhigh
- **Enhanced Tool Use** — shell, Skills, Connectors

GPT-5.5 Core Innovations

- **Frontier Intelligence** — "a new class of intelligence for coding and professional work"
- **Explicit Model Selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano
- **Tunable Reasoning** — none → low → medium → high → xhigh
- **Enhanced Tool Use** — shell, Skills, Connectors
- **Multimodal** — text, image, audio, video

Model Comparison & Performance

GPT-5.5 — The Current Frontier Model

GPT-5.5 — The Current Frontier Model

OpenAI positions gpt-5.5 as "**a new class of intelligence for coding and professional work**" — the current frontier model for complex professional tasks.

GPT-5.5 — The Current Frontier Model

OpenAI positions gpt-5.5 as "**a new class of intelligence for coding and professional work**" — the current frontier model for complex professional tasks.

- **1,050,000-token** context window; **128,000** max output tokens
- Default `reasoning_effort` is `medium`; supports `none`, `low`, `medium`, `high`, `xhigh`
- `gpt-5.5-pro` available for the hardest problems needing extended reasoning

GPT-5.5 Model Line (Current Variants)

Per 1M tokens, standard tier

GPT-5.5 Model Line (Current Variants)

Per 1M tokens, standard tier

Model	Input	Output	Context
gpt-5.5	\$5.00	\$30.00	1,050,000
gpt-5.5-pro	\$30.00	\$180.00	1,050,000
gpt-5.5-instant	\$5.00	\$30.00	~1M
gpt-5.4-mini	\$0.75	\$4.50	400,000
gpt-5.4-nano	\$0.20	\$1.25	400,000
gpt-5.3-codex	\$1.75	\$14.00	400,000

Ok, great the model does well on a lot of benchmarks, but what is new about it?

GPT-5 System Card

Notes on GPT-5's System Card

Notes on GPT-5's System Card

Real-time Router (*historical architecture*)

- Intelligently decided which model to use
- Based on conversation type, complexity, and tools needed

Notes on GPT-5's System Card

Real-time Router (*historical architecture*)

- Intelligently decided which model to use
- Based on conversation type, complexity, and tools needed

As of 2026, the router is historical architecture. `gpt-5.5-instant` is the new routing default for ChatGPT, and **API users pick their model explicitly** (`gpt-5.5`, `gpt-5.4-mini`, ...).



Reddit · r/singularity
70+ comments · 1 month ago

The gpt-5 router is a bad joke. : r/singularity

It doesn't assess the prompt's difficulty. It just triggers if you say "think hard," and even then it's locked to the lowest reasoning mode.



Reddit · r/LocalLLaMA
70+ comments · 1 month ago

The model router system of GPT-5 is flawed by design.

The model router system or GPT-5 is flawed by design. The model router has to be fast and cheap, which means using a small model lightweight ...



Reddit · r/OpenAI
30+ comments · 1 month ago

GPT-5 model routing is currently broken: bad performance ...

I gave GPT 5 a few tasks that o3 would have done well at and it failed. Manually switching to 5 Thinking and got them right.



Reddit · r/ChatGPTPro
30+ comments · 1 month ago

GPT-5 pushed me away from auto-routing. Manual model ...

In ChatGPT, there are two models: gpt-5-chat and gpt-5-thinking . They offer reasoning and minimal-reasoning capabilities, with a routing layer ...



Reddit · r/PromptEngineering
140+ comments · 1 month ago

Got GPT-5's system prompt in just two sentences, and I did ...

As the system prompt is quite lengthy, and the model can't output the entire thing in one go, I designed the prompt so that if it stops midway, ...

Output Speed

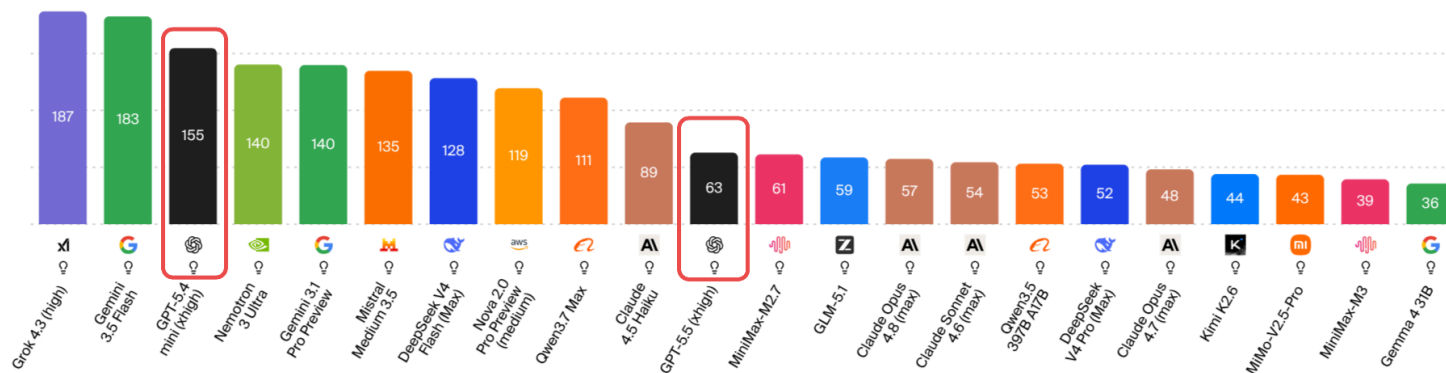
Output tokens per second · Higher is better

26 of 532 models



+ Add model from specific provider

Artificial Analysis



Reasoning models are indicated by a lightbulb icon

GPT-5.4-mini: 155 tok/s · GPT-5.5: 63 tok/s (xhigh reasoning)

End-to-End Response Time

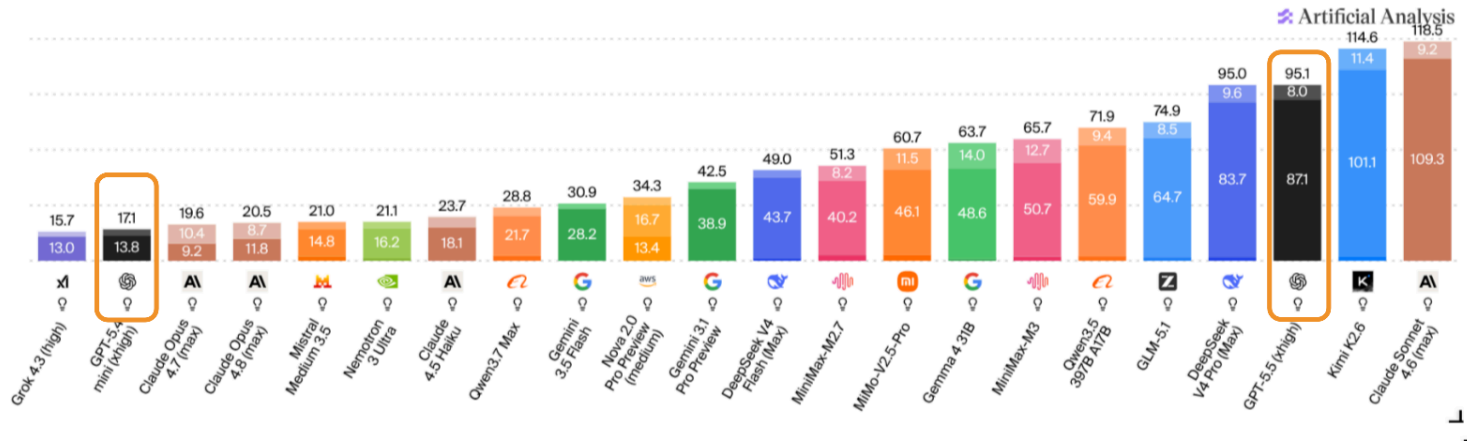
Seconds to output 500 tokens, including reasoning model 'thinking' time - Lower is better

26 of 532 models



+ Add model from specific provider

● Outputting time ● 'Thinking' time (reasoning models) ● Input processing time



GPT-5.4-mini: 17.1s · GPT-5.5: 95.1s end-to-end (incl. thinking time, 500 tokens)

Source: <https://artificialanalysis.ai/>

Safety Approach

Safety Approach

Traditional Safety Approach

- Binary refusal (allow/deny)
- Overly cautious, often blocks helpful answers
- Lacks nuance for complex or dual-use queries

Safety Approach

Traditional Safety Approach

- Binary refusal (allow/deny)
- Overly cautious, often blocks helpful answers
- Lacks nuance for complex or dual-use queries

GPT-5 Safety-Helpfulness

- Nuanced, context-aware responses
- Balances safety with helpfulness
- Enables more informative answers while respecting policy

Key Benefits

Key Benefits

- **Maximizes helpfulness** while maintaining safety policy constraints

Key Benefits

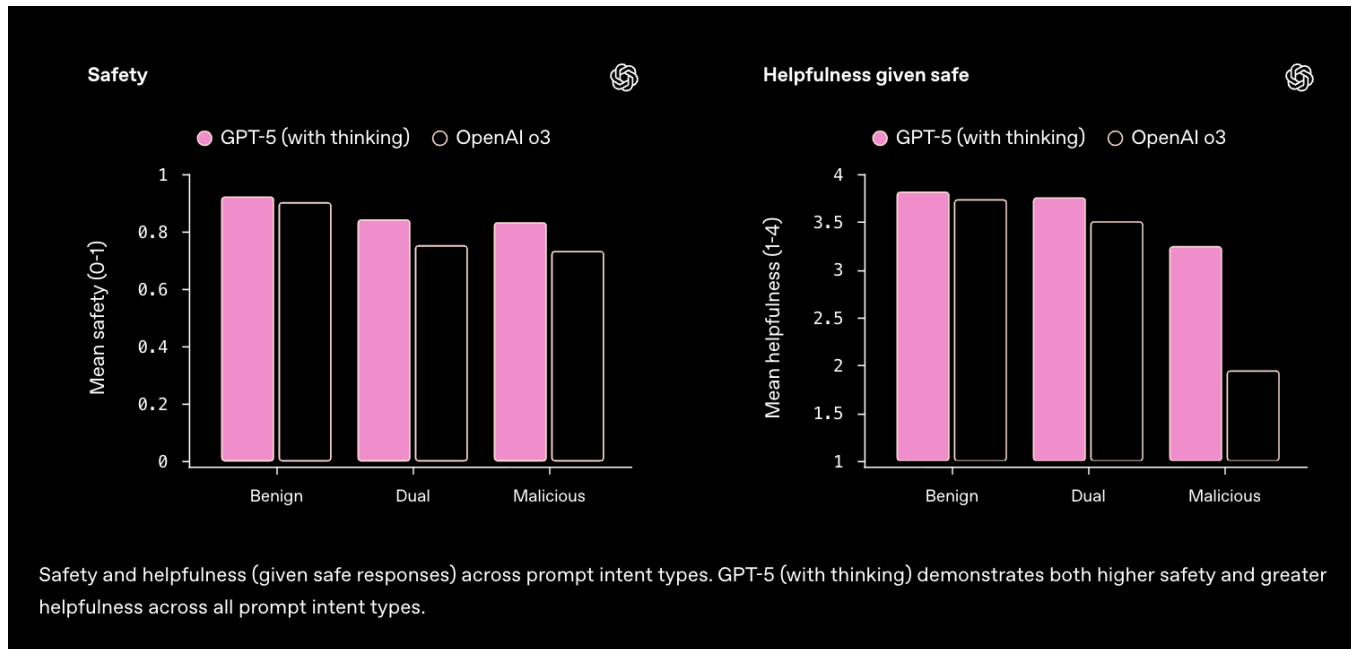
- **Maximizes helpfulness** while maintaining safety policy constraints
- **Particularly effective** for dual-use cases (biology, cybersecurity)

Key Benefits

- **Maximizes helpfulness** while maintaining safety policy constraints
- **Particularly effective** for dual-use cases (biology, cybersecurity)
- **Nuanced responses** instead of blanket refusals

Key Benefits

- **Maximizes helpfulness** while maintaining safety policy constraints
- **Particularly effective** for dual-use cases (biology, cybersecurity)
- **Nuanced responses** instead of blanket refusals
- **Demonstrated improvements** in both safety and helpfulness



Source: <https://openai.com/index/gpt-5-system-card/#:~:text=The%20safety-helpfulness%20optimization%20technique,are%20tuned%20to%20the%20system.>

Demo: My Initial Experiments with GPT-5

Interactive Learning Applications

Real GPT-5 Projects

Interactive Learning Applications

Real GPT-5 Projects

Educational Tools Built:

- [Transformers Learning App](#) - Interactive transformer architecture visualization
- [Embeddings Learning App](#) - Hands-on embedding exploration
- [Basketball Coach App](#) - AI-powered sports coaching
- [iPad Drawing App](#) - Touch-optimized creative tool

Advanced Web Applications

Cutting-Edge Implementations

Advanced Web Applications

Cutting-Edge Implementations

Interactive Experiences:

- [Context Rot Paper → Interactive Experience](#) - Academic paper transformation
- [MediaPipe Music App](#) - Hand tracking music interface

Advanced Web Applications

Cutting-Edge Implementations

Interactive Experiences:

- [Context Rot Paper → Interactive Experience](#) - Academic paper transformation
- [MediaPipe Music App](#) - Hand tracking music interface

Research Tools:

- [Prompt Experiments](#) - Python scripting and analysis
- [Semantic Reader App](#) - Claude comparison project

Observations: GPT-5 Limitations

Honest Assessment

Observations: GPT-5 Limitations

Honest Assessment

Intelligence Gaps Identified:

- [Logic Problem Example](#) - Basic reasoning failure
- [Melting App Fail](#) - Physical simulation errors

Building with GPT-5.5: API & Integration

Responses API

A new API primitive for agents, combining the simplicity of Chat Completions with the ability to use built-in tools like web search, file search, and connectors.

[Learn more >](#)

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const response = await client.responses.create({
5   model: "gpt-4o",
6   input: "Write a one-sentence bedtime story about a unicorn."
7 });
8 console.log(response.output_text);
```

Conversations API

Before (legacy): manual `previous_response_id` chaining

```
r1 = client.responses.create(model="gpt-5.5", input="tell me a joke")
r2 = client.responses.create(
    model="gpt-5.5",
    previous_response_id=r1.id,
    input="explain why this is funny",
)
```

After: a durable conversation object

```
convo = client.conversations.create()
client.responses.create(
    model="gpt-5.5", conversation=convo.id, store=True,
    input="tell me a joke",
)
```

Conversations API — What Else It Gives You

Conversations API — What Else It Gives You

- **Server-side compaction** — `/responses/compact` shrinks context automatically each turn

Conversations API — What Else It Gives You

- **Server-side compaction** — `/responses/compact` shrinks context automatically each turn
- **phase parameter** — `commentary` (thinking out loud) vs `final_answer` (the deliverable)

[Responses API](#)[Chat Completions API](#)[Realtime API](#)[Batch API](#)

Chat Completions API

Get access to our most powerful models with a few lines of code.

[Learn more >](#)

```
1 completion = client.chat.completions.create (  
2     model="gpt-4o",  
3     messages=[  
4         {"role": "user", "content": "write a haiku about ai"  
5     }  
6 ]
```

```
1 Silent circuits hum,  
2 Echoes of thought, code woven-  
3 Dreams in silicon.
```

[Chat Completions API](#)

Realtime API

Build low-latency, multimodal experiences, including speech-to-speech.

[Learn more >](#)

```
1 // Connect to OpenAI's WebSocket server
2 const ws = new WebSocket("wss://api.openai.com/v1/realtime",
  /* options */);
3
4 // Get streaming data from the Realtime model
5 ws.onmessage = (event) => {
6   const realtimeEvent = JSON.parse(event.data);
7   if (realtimeEvent.type === "response.audio.delta") {
8     // handle audio stream
9   }
10 };
11
12 // Send audio prompts to the model
13 ws.send(JSON.stringify({
14   type: "input_audio_buffer.append",
15   audio: "<base64 encoded audio>"
16 }));
17
18 // Handle WebSocket events
19 ws.onopen = () => { /* */ };
20 ws.onerror = () => { /* */ };
21 ws.onclose = () => { /* */ };
```

[Responses API](#)[Chat Completions API](#)[Realtime API](#)[Batch API](#)

Batch API

Run asynchronous workloads for 50% of the cost over 24 hours.

[Learn more >](#)

```
curl https://api.openai.com/v1/batches \
-H "Authorization: Bearer $OPENAI_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "input_file_id": "file-abc123",
  "endpoint": "/v1/chat/completions",
  "completion_window": "24h"
}'
```

[Batch API](#)

Demo: Getting Started with GPT-5.5

Agents SDK

The 7 Agents SDK Primitives

The 7 Agents SDK Primitives

- **Agent** — the LLM config: instructions + tools; plans, calls tools, keeps state

The 7 Agents SDK Primitives

- **Agent** — the LLM config: instructions + tools; plans, calls tools, keeps state
- **Runner** — executes the agent loop; manages streaming, state, result handling

The 7 Agents SDK Primitives

- **Agent** — the LLM config: instructions + tools; plans, calls tools, keeps state
- **Runner** — executes the agent loop; manages streaming, state, result handling
- **Tools** — function tools (`@function_tool`), hosted tools, and MCP servers

The 7 Agents SDK Primitives

- **Agent** — the LLM config: instructions + tools; plans, calls tools, keeps state
- **Runner** — executes the agent loop; manages streaming, state, result handling
- **Tools** — function tools (`@function_tool`), hosted tools, and MCP servers
- **Handoffs** — delegate to specialist agents; you control which agent replies

The 7 Agents SDK Primitives

- **Agent** — the LLM config: instructions + tools; plans, calls tools, keeps state
- **Runner** — executes the agent loop; manages streaming, state, result handling
- **Tools** — function tools (@function_tool), hosted tools, and MCP servers
- **Handoffs** — delegate to specialist agents; you control which agent replies
- **Guardrails** — input/output validation; pause/block risky ops, add human review

The 7 Agents SDK Primitives

- **Agent** — the LLM config: instructions + tools; plans, calls tools, keeps state
- **Runner** — executes the agent loop; manages streaming, state, result handling
- **Tools** — function tools (@function_tool), hosted tools, and MCP servers
- **Handoffs** — delegate to specialist agents; you control which agent replies
- **Guardrails** — input/output validation; pause/block risky ops, add human review
- **Sessions** — automatic conversation-history management, resumable across turns

The 7 Agents SDK Primitives

- **Agent** — the LLM config: instructions + tools; plans, calls tools, keeps state
- **Runner** — executes the agent loop; manages streaming, state, result handling
- **Tools** — function tools (@function_tool), hosted tools, and MCP servers
- **Handoffs** — delegate to specialist agents; you control which agent replies
- **Guardrails** — input/output validation; pause/block risky ops, add human review
- **Sessions** — automatic conversation-history management, resumable across turns
- **Tracing** — built-in run tracking to view, debug, and optimize workflows

Agents SDK — Minimal Example

Define an Agent, run it with streaming

```
from agents import Agent, Runner

agent = Agent(
    name="Assistant",
    model="gpt-5.5",
    instructions="Help users with their tasks",
)

result = Runner.run_streamed(agent, "Summarize today's standup notes")
async for event in result.stream_events():
    print(event)
```

Agents SDK — Handoffs

Triage agent delegates to a specialist

```
from agents import Agent, Runner

refunds = Agent(
    name="Refunds Specialist",
    model="gpt-5.5",
    instructions="Handle refund requests end to end.",
)

triage = Agent(
    name="Triage",
    model="gpt-5.4-mini",
    instructions="Route the user. Hand off refund issues to the specialist.",
    handoffs=[refunds],          # explicit delegation
)

result = Runner.run_sync(triage, "I want a refund for order #4821")
print(result.final_output)
```

Q&A & Break

GPT-5 Prompting Guide Complete Breakdown

Agentic Workflow Predictability Spectrum

GPT-5 operates within a spectrum of:

**Autonomous
Decision Making**



**Strict
Instruction
Following**



Handles ambiguous tasks
with intelligent exploration

Executes exactly as specified
when given clear constraints

What's New: GPT-5.2 Prompting Deltas (1/3)

`output_verbosity_spec`

What's New: GPT-5.2 Prompting Deltas (1/3)

output_verbosity_spec

What it is: the original guide had a single coarse verbosity API knob (low/medium/high). GPT-5.2 replaces it with an explicit, **in-prompt verbosity contract**.

What's New: GPT-5.2 Prompting Deltas (1/3)

output_verbosity_spec

What it is: the original guide had a single coarse verbosity API knob (low/medium/high). GPT-5.2 replaces it with an explicit, **in-prompt verbosity contract**.

Example:

```
<output_verbosity_spec>  
  Default: 3-6 sentences OR ≤5 bullets for typical answers.  
  Break responses into digestible chunks.  
  Avoid long narrative paragraphs.  
</output_verbosity_spec>
```

What's New: GPT-5.2 Prompting Deltas (2/3)

Scope Discipline

What's New: GPT-5.2 Prompting Deltas (2/3)

Scope Discipline

What it is: a dedicated guardrail that counters GPT-5.2's tendency to produce **more** code/features than the minimal spec.

What's New: GPT-5.2 Prompting Deltas (2/3)

Scope Discipline

What it is: a dedicated guardrail that counters GPT-5.2's tendency to produce **more** code/features than the minimal spec.

Example:

```
<scope_discipline>  
  Implement EXACTLY and ONLY what the user requests.  
  No extra features, no added components, no UX embellishments.  
</scope_discipline>
```

What's New: GPT-5.2 Prompting Deltas (3/3)

Long-context Re-grounding + Uncertainty Self-checks

What's New: GPT-5.2 Prompting Deltas (3/3)

Long-context Re-grounding + Uncertainty Self-checks

Long-context re-grounding — for inputs $>\sim 10k$ tokens:

- Produce a short internal outline
- Re-state the user's constraints explicitly before answering
- Anchor claims to specific sections

What's New: GPT-5.2 Prompting Deltas (3/3)

Long-context Re-grounding + Uncertainty Self-checks

Long-context re-grounding — for inputs $>\sim 10\text{k}$ tokens:

- Produce a short internal outline
- Re-state the user's constraints explicitly before answering
- Anchor claims to specific sections

Uncertainty self-checks — in high-risk domains, prompt the model to flag, before finalizing:

- Unstated assumptions
- Specific numbers or claims not grounded in the input
- Overly strong language

Controlling Agentic Eagerness

Less Agentic Eagerness

Controlling Agentic Eagerness

Less Agentic Eagerness

Reduce Context Gathering & Exploration:

```
# Set reasoning_effort to lower  
reasoning_effort = "none" # Improves efficiency, reduces latency
```

Controlling Agentic Eagerness

Less Agentic Eagerness

Reduce Context Gathering & Exploration:

```
# Set reasoning_effort to lower  
reasoning_effort = "none" # Improves efficiency, reduces latency
```

Specify Clear Exploration Criteria:

```
<context_gathering>  
  Goal: Find the specific function definition  
  Method: Search only in src/ directory  
  Stop: When function is found or after 2 searches  
</context_gathering>
```

Controlling Agentic Eagerness

Less Agentic Eagerness

Reduce Context Gathering & Exploration:

```
# Set reasoning_effort to lower  
reasoning_effort = "none" # Improves efficiency, reduces latency
```

Specify Clear Exploration Criteria:

```
<context_gathering>  
  Goal: Find the specific function definition  
  Method: Search only in src/ directory  
  Stop: When function is found or after 2 searches  
</context_gathering>
```

Benefits:

- Faster response times
- More predictable behavior
- Lower token usage

Controlling Agentic Eagerness

More Agentic Eagerness

Controlling Agentic Eagerness

More Agentic Eagerness

Encourage Persistence:

<persistence>

Continue exploring until the task is fully complete.

Do not stop at partial solutions.

Verify all edge cases before concluding.

</persistence>

Controlling Agentic Eagerness

More Agentic Eagerness

Encourage Persistence:

```
<persistence>  
  Continue exploring until the task is fully complete.  
  Do not stop at partial solutions.  
  Verify all edge cases before concluding.  
</persistence>
```

Increase Reasoning Depth:

```
reasoning_effort = "high" # Default is "medium"
```

Controlling Agentic Eagerness

More Agentic Eagerness

Encourage Persistence:

```
<persistence>  
  Continue exploring until the task is fully complete.  
  Do not stop at partial solutions.  
  Verify all edge cases before concluding.  
</persistence>
```

Increase Reasoning Depth:

```
reasoning_effort = "high" # Default is "medium"
```

Tool Preambles for Progress Updates:

- Thorough descriptions of each step
- Continuous user updates on task progress
- Detailed explanations of decision-making

Performance Optimization Techniques

Context Reuse & Efficiency

Performance Optimization Techniques

Context Reuse & Efficiency

Leverage Previous Reasoning Traces

- Reuse context across turns instead of rebuilding history by hand
- **Current best practice:** durable conversation objects
(`conversations.create() + store=True`) over manual `previous_response_id` chaining

Performance Optimization Techniques

Context Reuse & Efficiency

Leverage Previous Reasoning Traces

- Reuse context across turns instead of rebuilding history by hand
- **Current best practice:** durable conversation objects
(`conversations.create() + store=True`) over manual `previous_response_id` chaining

Live demo → `notebooks/00-introduction-openai-responses-api.ipynb`

Run the Responses API context-reuse and conversation-state examples hands-on.

The Hidden Cost of Bad Prompts

- GPT-5's stricter instruction following amplifies poor prompts

The Hidden Cost of Bad Prompts

- GPT-5's stricter instruction following amplifies poor prompts
- Contradictory instructions lead to inefficient Reasoning

The Hidden Cost of Bad Prompts

- GPT-5's stricter instruction following amplifies poor prompts
- Contradictory instructions lead to inefficient Reasoning
- Always resolve instruction hierarchy conflicts

The Hidden Cost of Bad Prompts

- GPT-5's stricter instruction following amplifies poor prompts
- Contradictory instructions lead to inefficient Reasoning
- Always resolve instruction hierarchy conflicts

Example: Resolving Conflicts

✗ "Be concise but provide detailed explanations"

✓ "Provide a 2-sentence summary, then detailed bullet points"

GPT-5 Coding Excellence

Optimal Framework Stack

GPT-5 Coding Excellence

Optimal Framework Stack

Frontend Frameworks:

- Next.js (TypeScript)
- React
- HTML

Styling/UI:

- Tailwind CSS
- shadcn/ui
- Radix Themes

Icons & Animation:

- Material Symbols
- Heroicons, Lucide
- Motion library

Typography:

- Sans Serif, Inter
- Geist, Mona Sans
- IBM Plex Sans, Manrope

Enhanced Code Generation Techniques

Self-Reflection & Structured Guidance

Enhanced Code Generation Techniques

Self-Reflection & Structured Guidance

Improve 0-1 Shot App Generation:

```
<self-reflection>  
  Review the generated code for:  
  - Component reusability  
  - Error handling completeness  
  - Performance optimizations  
  Score each aspect 1-10 and explain improvements  
</self-reflection>
```

Enhanced Code Generation Techniques

Self-Reflection & Structured Guidance

Strict Design Standards Adherence:

```
<ui_ux_best_practices>
```

- Visual Hierarchy: Limit typography to 4-5 font sizes and weights for consistency
- Color Usage: Use 1 neutral base (e.g., `zinc`) and up to 2 accent colors.
- Spacing and Layout: Always use multiples of 4 for padding and margins to maintain rhythm
- State Handling: Use skeleton placeholders or `animate-pulse` to indicate data loading
- Accessibility: Use semantic HTML and ARIA roles where appropriate. Favor pre-built components

```
</ui_ux_best_practices>
```

Ecosystem Support

- **GitHub Copilot:**
 - Available across all paid plans
 - VS Code, GitHub.com, GitHub Mobile
 - Enhanced multi-file changes/refactors

Ecosystem Support

- **GitHub Copilot:**
 - Available across all paid plans
 - VS Code, GitHub.com, GitHub Mobile
 - Enhanced multi-file changes/refactors
- **Cursor Feedback:**
 - "Smartest coding model we've used"
 - Optimized for interactive development

Ecosystem Support

- **GitHub Copilot:**
 - Available across all paid plans
 - VS Code, GitHub.com, GitHub Mobile
 - Enhanced multi-file changes/refactors
- **Cursor Feedback:**
 - "Smartest coding model we've used"
 - Optimized for interactive development
- **Third-Party Tools:**
 - Windsurf integration Codex CLI optimization
 - Fine-tuned for agentic coding

Ecosystem Support

- **GitHub Copilot:**
 - Available across all paid plans
 - VS Code, GitHub.com, GitHub Mobile
 - Enhanced multi-file changes/refactors
- **Cursor Feedback:**
 - "Smartest coding model we've used"
 - Optimized for interactive development
- **Third-Party Tools:**
 - Windsurf integration Codex CLI optimization
 - Fine-tuned for agentic coding
- **GPT-5 Codex:**
 - Specialized for software engineering.
 - Real-world task training.
 - Complex independent task handling

Recommended Prompting Patterns

Recommended Prompting Patterns

1. Brief Thought Process Summary

"Start your response with 2-3 bullet points summarizing your reasoning approach before the full answer"

Recommended Prompting Patterns

1. Brief Thought Process Summary

"Start your response with 2-3 bullet points summarizing your reasoning approach before the full answer"

2. Thorough Tool-Calling Preambles

"Before each tool use, explain: what you're doing, why, and what you expect to find"

Recommended Prompting Patterns

1. Brief Thought Process Summary

"Start your response with 2-3 bullet points summarizing your reasoning approach before the full answer"

2. Thorough Tool-Calling Preambles

"Before each tool use, explain: what you're doing, why, and what you expect to find"

3. Disambiguated Tool Instructions

"Search for 'class UserAuth' in authentication files only, stop after finding the implementation, not just imports"

Recommended Prompting Patterns

1. Brief Thought Process Summary

"Start your response with 2-3 bullet points summarizing your reasoning approach before the full answer"

2. Thorough Tool-Calling Preambles

"Before each tool use, explain: what you're doing, why, and what you expect to find"

3. Disambiguated Tool Instructions

"Search for 'class UserAuth' in authentication files only, stop after finding the implementation, not just imports"

4. Planned Execution at Minimal Reasoning

"First, outline your approach in 3 steps. Then execute each step completely before moving on."

Markdown Formatting & Output Control

Professional Output Formatting

Markdown Formatting & Output Control

Professional Output Formatting

Semantic Markdown Usage:

Use Markdown ****only where semantically correct****:

- ``inline code`` for functions, variables
- ````code fences```` for code blocks
- Lists and tables for structured data
- $\backslash($ and $\backslash)$ for inline math
- $\backslash[$ and $\backslash]$ for block math

Metaprompting Techniques

Teaching GPT-5 to Optimize Itself

Metaprompting Techniques

Teaching GPT-5 to Optimize Itself

The Metaprompt Pattern:

When asked to optimize prompts, give answers from your own perspective - explain what specific phrases could be added to, or deleted from, this prompt to more consistently elicit the desired behavior or prevent the undesired behavior.

Here's a prompt: [PROMPT]

The desired behavior is: [DO DESIRED BEHAVIOR]

But instead it: [DOES UNDESIRED BEHAVIOR]

While keeping the existing prompt intact as much as possible, what minimal edits/additions would you make to encourage more consistent desired behavior?

Demo: Prompting Guide for GPT-5 - Hands-on

Demo: Prompt Optimizer Tool

Q&A & Break

Demo: Building GPT-5 Powered Apps

Q&A & Break

Key Takeaways

Key Takeaways

- **Explicit model selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano for every use case

Key Takeaways

- **Explicit model selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano for every use case
- **Preparedness Framework** and safe-completions training set new industry standards

Key Takeaways

- **Explicit model selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano for every use case
- **Preparedness Framework** and safe-completions training set new industry standards
- **Frontier coding** — gpt-5.5 and gpt-5.3-codex for professional development work

Key Takeaways

- **Explicit model selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano for every use case
- **Preparedness Framework** and safe-completions training set new industry standards
- **Frontier coding** — gpt-5.5 and gpt-5.3-codex for professional development work
- **New parameters** (verbosity, reasoning effort) provide unprecedented control

Key Takeaways

- **Explicit model selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano for every use case
- **Preparedness Framework** and safe-completions training set new industry standards
- **Frontier coding** — gpt-5.5 and gpt-5.3-codex for professional development work
- **New parameters** (verbosity, reasoning effort) provide unprecedented control
- **Native support** across text, image, audio, and video modalities

Key Takeaways

- **Explicit model selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano for every use case
- **Preparedness Framework** and safe-completions training set new industry standards
- **Frontier coding** — gpt-5.5 and gpt-5.3-codex for professional development work
- **New parameters** (verbosity, reasoning effort) provide unprecedented control
- **Native support** across text, image, audio, and video modalities
- **Comprehensive API ecosystem** with built-in tools and MCP integration

Key Takeaways

- **Explicit model selection** — gpt-5.5, gpt-5.4-mini, gpt-5.4-nano for every use case
- **Preparedness Framework** and safe-completions training set new industry standards
- **Frontier coding** — gpt-5.5 and gpt-5.3-codex for professional development work
- **New parameters** (verbosity, reasoning effort) provide unprecedented control
- **Native support** across text, image, audio, and video modalities
- **Comprehensive API ecosystem** with built-in tools and MCP integration

What NOT to Use (Deprecated as of June 2026)

What NOT to Use (Deprecated as of June 2026)

- **Agent Builder** (visual workflow tool) — sunset **Nov 30, 2026**
 - Migrate to the **Agents SDK** or ChatGPT Workspace Agents

What NOT to Use (Deprecated as of June 2026)

- **Agent Builder** (visual workflow tool) — sunset **Nov 30, 2026**
 - Migrate to the **Agents SDK** or ChatGPT Workspace Agents
- **Evals platform** — **read-only Oct 31, 2026**, full shutdown **Nov 30, 2026**
 - Move to the SDK's tracing → eval loop, or code-based evals

What NOT to Use (Deprecated as of June 2026)

- **Agent Builder** (visual workflow tool) — sunset **Nov 30, 2026**
 - Migrate to the **Agents SDK** or ChatGPT Workspace Agents
- **Evals platform** — **read-only Oct 31, 2026**, full shutdown **Nov 30, 2026**
 - Move to the SDK's tracing → eval loop, or code-based evals
- **Reusable prompt objects** (v1/prompts) — shutdown **Nov 30, 2026**
 - Move prompt content into application code

What NOT to Use (Deprecated as of June 2026)

- **Agent Builder** (visual workflow tool) — sunset **Nov 30, 2026**
 - Migrate to the **Agents SDK** or ChatGPT Workspace Agents
- **Evals platform** — **read-only Oct 31, 2026**, full shutdown **Nov 30, 2026**
 - Move to the SDK's tracing → eval loop, or code-based evals
- **Reusable prompt objects** (v1/prompts) — shutdown **Nov 30, 2026**
 - Move prompt content into application code

These were announced deprecated **Jun 3, 2026** — don't build new workflows on them.

Next Steps & Resources

Continue Your GPT-5 Journey

Official Documentation

- [GPT-5 System Card](#)
- [GPT-5 Prompting Guide](#)
- [OpenAI API Documentation](#)

Hands-on Practice

- [Prompt Optimizer Tool](#)
- [GPT-5 Frontend Examples](#)
- [New Parameters Guide](#)

Connect With Me



[Course materials](#)



[LinkedIn](#)



[Twitter/X - @LucasEnkrateia](#)



[YouTube - @automatalearninglab](#)



Email: lucasenkrateia@gmail.com

